Relatório: Folhagem & Cia

Nome: Alam | Branch: `desafio/alam`

1. O que foi feito

Terminei as 7 rotas obrigatórias e mais 1 de bônus, ou seja, 8 no total. Testei todas no Thunder Client, seguindo a tabela da parte 4 do PDF, e todas passaram. São elas: `POST /plantas`, `GET /plantas` (que aceita os filtros `?luminosidade=` e `?petFriendly=`, sozinhos ou juntos), `GET /plantas/:id`, `PATCH /plantas/:id`, `DELETE /plantas/:id`, `POST /plantas/:id/venda`, `GET /plantas/estoque/valor-total` e `GET /plantas/esgotadas` (bônus).

As regras de negócio também estão todas cumpridas. No cadastro todos os campos são obrigatórios. O `preco` tem que ser inteiro maior que 0, o `estoque` inteiro maior ou igual a 0 (zero vale, é planta esgotada), a `regaACadaDias` maior ou igual a 1 e a `luminosidade` só aceita os valores do enum. Na venda, a `quantidade` precisa ser inteira e maior que 0. Dado inválido no cadastro, na edição, nos filtros ou na venda volta 400 com mensagem, id que não existe volta 404, e a venda nunca deixa o estoque negativo: se não tem estoque suficiente, responde 400 e não altera nada.

Dos bônus, fiz a busca por nome (`?nome=`, que não diferencia maiúscula de minúscula), a ordenação (`?ordem=preco` ou `?ordem=nome`, esta em ordem alfabética) e a rota `GET /plantas/esgotadas`. O script de seed ficou de fora.

2. Como foi feito

Fui seguindo os passos do enunciado, com um commit por etapa: model e migration da `Planta`, `PlantaRepository`, `PlantaController`, rotas, filtros, valor total e, por último, a venda. Depois de cada etapa eu testava no Thunder Client antes de ir para a próxima. Os bônus só vieram no fim, depois que o obrigatório estava completo. Para a organização usei o CRUD de usuário como molde (rotas, controller, repository e banco), onde só o repository conversa com o Prisma.

Algumas decisões que tomei pelo caminho, e por quê:

- **Preço em centavos:** o enunciado explica que decimal dá erro de arredondamento (0.1 + 0.2), e com inteiro a conta de dinheiro sempre fecha.
- **`=== undefined` em vez de `!campo`:** o `!campo` trata `0` e `false` como se o campo estivesse vazio, então recusaria um `estoque: 0` ou um `petFriendly: false`, que são válidos. Comparar com `undefined` pega só o que realmente não foi enviado.
- **Uma função de validação para POST e PATCH:** a `validarPlanta(body, parcial)` exige todos os campos no cadastro e, na edição, só valida os que vieram. Assim as regras ficam escritas uma vez só.
- **Buscar antes de editar, deletar e vender:** se a planta não existe, o Prisma joga um erro que cairia no `catch` e viraria um 500 errado. Buscando antes, consigo responder 404.
- **`return` em todo `res.status` dentro de `if`:** sem ele a função continua rodando e dá o erro "Cannot set headers after they are sent".
- **Desestruturar o `req.body`:** mando para o banco só os campos da planta, nunca o body inteiro.
- **Filtros:** o `?petFriendly=` chega como texto, então converto com `=== "true"`, mas deixando `undefined` quando o filtro não foi enviado (senão virava `false` e filtrava sem ninguém pedir). Como o

Prisma ignora condição `undefined`, uma consulta só cobre todas as combinações. Filtro inválido dá 400, e filtro válido sem resultado dá 200 com `[]`.

- **`/plantas/esgotadas` antes de `/plantas/:id`:** o Express usa a primeira rota que combina, então se fosse depois o "esgotadas" seria tratado como id.

Fiz também algumas coisas além do que o enunciado pedia. O id inválido (tipo `abc`) retorna 400 também no PATCH, no DELETE e na venda, para o `NaN` não chegar no Prisma e virar 500. E `?nome=` vazio ou `?ordem=` diferente de `preco`/`nome` também retornam 400.

Sobre a venda, usei a versão simples: valido, busco a planta, confiro se `estoque >= quantidade` e só então decremento. O enunciado perguntava o que acontece se duas pessoas compram o último vaso ao mesmo tempo, e pensando nisso vi que essa versão tem uma *race condition*: as duas requisições podem ler `estoque: 1` antes de qualquer uma decrementar, e o estoque fica negativo. Deixei assim de propósito, porque é mais simples de explicar, e deixei um comentário no código avisando.

3. Principais dificuldades

Boa parte do desafio foi a primeira vez que eu fazia cada coisa, então as dificuldades vieram menos de erro de digitação e mais de entender o que estava acontecendo.

Instalar o Docker. O instalador falhou dizendo que faltava o recurso "Virtual Machine Platform" do Windows e que a instalação por usuário não consegue ativá-lo. Resolvi abrindo o PowerShell como administrador, rodando `wsl --install` e reiniciando o PC.

PowerShell bloqueando o npm. O `npm -v` foi recusado por causa da política de execução de scripts. Resolvi com `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` e depois instalei o Yarn.

O erro 500 na primeira planta. O request estava certo, mas a API devolvia 500. Percebi que a mensagem vinha do `catch` do controller, então fui olhar o erro de verdade em `docker compose logs`. O log mostrava o `Object.values` recebendo `undefined`: o Prisma Client que roda dentro do container tinha sido gerado antes do enum `Luminosidade` existir e não atualizou sozinho. Resolvi com `docker compose restart server`. Aprendi que, quando mexo no `schema.prisma`, preciso rodar o `yarn migration` na minha máquina **e** reiniciar o container.

Aprender JavaScript/TypeScript na prática. Foi meu primeiro contato com a linguagem, e o difícil foi entender o que cada parte do código fazia, e não só copiar. O que mais me travou foi a validação: entender por que `!campo` recusa `0` e `false` e por que o certo é comparar com `undefined`. Também precisei entender como o `body` chega no código, por que ele pode vir vazio (`req.body ?? {}`) e por que eu pego campo por campo em vez de mandar o `req.body` direto para o banco. Os tipos (`string`, `number`, `boolean`) ajudaram a ver o que cada função espera, e o `yarn typecheck` me mostrou erros antes mesmo de eu testar.

Entender a organização em camadas. Eu nunca tinha visto um projeto dividido em rotas, controller e repository, e no começo não fazia sentido separar tanto. Lendo o CRUD de usuário fui entendendo o papel de cada um: a rota liga a URL à função, o controller lê, valida e escolhe o status, e o repository é o único que fala com o banco. Aí os status também fizeram sentido: 201 para criado, 204 para removido, 400 para dado inválido, 404 para id inexistente e 500 para erro do servidor.

Entender a comunicação com o banco. Foi a primeira vez que mexi com banco de dados. Precisei entender que o `schema.prisma` descreve as tabelas, que a migration aplica isso no banco, e que a API roda dentro do Docker enquanto os comandos `yarn` rodam na minha máquina, o que explica o erro 500 de cima. Também

aprendi como o Prisma transforma funções em consultas: `where` para filtrar, `select` para pegar só alguns campos, `decrement` para diminuir o estoque, e que condição `undefined` é ignorada, o que deixa combinar filtros numa consulta só.

Venda e valor total. No valor total, o `aggregate` do Prisma não multiplica colunas, então busquei só `preco` e `estoque` e fiz a conta no código com um `for`. Conferi o resultado somando `preco × estoque` na mão e comparando com o banco. Na venda, o difícil foi pensar na ordem das checagens (id, quantidade, 404, estoque suficiente) e entender a race condition que expliquei na seção 2.

Interpretar as respostas dos testes. Aprendi a diferenciar o 404 "Rota não encontrada" (a URL não casou com nenhuma rota, ou a rota ainda não foi registrada) do 404 "Planta não encontrada" (a rota existe, mas o id não). Aprendi também a testar os casos de erro, e não só o caminho feliz, e a conferir os números na mão em vez de confiar só no que a API respondeu.

4. Uso de IA

Usei o **Claude (Anthropic)** do começo ao fim, como apoio para construir e também para aprender, junto com as aulas do treinamento.

Usei para:

- **Ambiente:** instalar Docker, Node e Yarn, rodar o projeto e entender os erros de instalação (WSL, PowerShell, Docker).
- **Código:** gerar a maior parte do esqueleto da `Planta` (repository, controller, rotas, filtros, venda, valor total e bônus), tendo como base o CRUD de usuário que já vinha no repositório e o enunciado. Antes disso li o código de usuário, o enunciado e assisti às aulas (principalmente git, GitHub e CRUD). A partir do esqueleto fui lendo e testando cada parte para entender como funciona.
- **Entender erros:** quando algo quebrava, como o 500 do Prisma Client, eu olhava o log, mandava o erro e usava a IA para me ajudar a interpretar a causa.
- **Testes:** pedi a lista de casos para testar e os resultados esperados, com base na parte 4 do enunciado.
- **Explicações:** usei como "professora" para conceitos que eu não conhecia, como `undefined`, o `body`, as camadas, os status HTTP e como o Prisma conversa com o banco.

Para conferir o que ela respondeu:

- Rodei todos os testes da parte 4 no Thunder Client, incluindo os de erro, e comparei cada status com o esperado.
- Conferi o valor total somando `preco × estoque` na mão. Quando deu diferente do que a IA tinha previsto, valeu a conta do banco (a previsão dela estava errada).
- Rodei o `yarn typecheck` e li o código para entender cada parte antes de commitar.
- Tomei decisões minhas: usar a versão simples da venda (mais fácil de explicar) e deixar o seed de fora.

No fim, aprendi como se organiza um CRUD em camadas, o que cada status HTTP significa, como validar a entrada (com o cuidado com `0` e `false`), como o `body` e a query chegam na API, como Prisma, migrations e Docker se conectam, como funciona a venda com a checagem de estoque (e a race condition que ela deixa aberta) e como investigar um erro lendo o log em vez de adivinhar. Foi meu primeiro contato com JavaScript/TypeScript, então a IA acelerou bastante o esqueleto e o meu foco foi entender o que estava sendo feito. Boa parte do código foi gerada com ajuda dela, e consigo explicar o que entreguei.